

MINISTERE DE L'ENSEIGNEMENT

MINISTERE DELEGUE CHARGE
L'ENSEIGNEMENT SUPERIEUR ET DE
LA RECHERCHE

REPUBLIQUE TOGOLAISE

Travail-Liberté-Patrie

Logo Ecole

Logo Entreprise

INSTITUT POLYTECHNIQUE DEFITECH

MEMOIRE

EN VUE DE L'OBTENTION DU DIPLOME DE LICENCE PROFESSIONNELLE

Domaine : Sciences et Technologies

Mention : Informatique

Spécialité : Génie Logiciel

Thème :

(Pas plus de 2 lignes)

Rédigé par :

Nom et Prénoms de l'étudiant

Maitre de Stage

Nom Prénoms

(Titre/Profession)

Directeur du Mémoire

Nom Prénoms

(Titre/Profession)

ANNEE ACADEMIQUE : 2025

Logo Ecole

Logo Entreprise

INSTITUT POLYTECHNIQUE DEFITECH

MEMOIRE
EN VUE DE L'OBTENTION DU DIPLOME DE
LICENCE PROFESSIONNELLE

Domaine : Sciences et Technologies

Mention : Informatique

Spécialité : Génie Logiciel

Thème :

Lorem

Présenté par : Nom et Prénoms de l'étudiant

Le(Date de la soutenance)

Directeur du Mémoire :

JURY

Président

Membre

Membre

ANNEE ACADEMIQUE : 2025

DEDICACES

Section facultative mais recommandée. L'apprenant dédie son travail à une ou plusieurs personnes qui lui sont chères (parents, famille, mentor). La dédicace est sobre, sincère et brève. Style centré ou aligné à droite. Longueur : 5 à 10 lignes maximum.

REMERCIEMENTS

Remercier dans l'ordre : les responsables de l'institution (directeur, responsable pédagogique), l'encadreur académique (directeur du rapport), le maître de stage et les responsables de la structure d'accueil, les membres du jury (formule générique), les collègues et camarades ayant contribué, la famille et les proches. Ton professionnel et reconnaissant. Longueur : 1 à 2 pages.

RESUME

Le résumé en français expose de manière synthétique : (1) le contexte et la problématique du projet logiciel, (2) les objectifs de développement fixés, (3) la méthodologie de développement adoptée (cycle en V, Agile/Scrum, Merise, UML...), (4) les principales fonctionnalités réalisées et les technologies utilisées (langages, frameworks, SGBD), (5) les résultats obtenus (livrable fonctionnel, performances, tests). Longueur : 150 à 250 mots. Terminer par 3 à 5 mots-clés.

Mots-clés : [Mot-clé 1], [Mot-clé 2], [Mot-clé 3], [Mot-clé 4], [Mot-clé 5]

ABSTRACT

The abstract is the English version of the résumé. It must faithfully cover: (1) context and problem statement, (2) development objectives, (3) software development methodology (Agile, Merise, UML...), (4) key features implemented and technologies used, (5) results obtained (functional deliverable, test outcomes). Length: 150 to 250 words. End with 3 to 5 keywords.

Keywords: [Keyword 1], [Keyword 2], [Keyword 3], [Keyword 4], [Keyword 5]

SOMMAIRE

Le sommaire liste les grands chapitres uniquement (niveau 1). Il est généré automatiquement par Word : Références → Table des matières → Table des matières personnalisée (niveau 1 uniquement). À placer avant la liste des figures.

LISTE DES FIGURES

Liste de toutes les figures du rapport : diagrammes UML (cas d'utilisation, classes, séquences, activités, déploiement), maquettes d'interface (wireframes, captures d'écran), schémas de base de données (MCD, MLD), diagrammes d'architecture. Format : Figure N : [Titre de la figure] p. XX. Générée automatiquement via Word (Références → Insérer une table des illustrations). Minimum attendu : 8 figures.

LISTE DES TABLEAUX

Liste de tous les tableaux : tableau des exigences fonctionnelles, tableau des cas d'utilisation, tableau de comparaison des technologies, tableau des tests, tableau du dictionnaire de données, etc. Format : Tableau N : [Titre du tableau] p. XX. Minimum attendu : 3 tableaux.

LISTE DES SIGLES ET DES ABREVIATIONS

Répertorier par ordre alphabétique tous les sigles, acronymes et abréviations du rapport.
Exemples : API : Application Programming Interface | CSS : Cascading Style Sheets | CRUD : Create, Read, Update, Delete | DAO : Data Access Object | HTML : HyperText Markup Language | HTTP : HyperText Transfer Protocol | IDE : Integrated Development Environment | JEE : Jakarta Enterprise Edition | JSON : JavaScript Object Notation | MCD : Modèle Conceptuel de Données | MLD : Modèle Logique de Données | MVC : Model-View-Controller | ORM : Object-Relational Mapping | PHP : PHP Hypertext Preprocessor | REST : Representational State Transfer | SGBD : Système de Gestion de Bases de Données | SQL : Structured Query Language | UML : Unified Modeling Language | URL : Uniform Resource Locator | XML : eXtensible Markup Language.

INTRODUCTION GENERALE

1. Contexte général

Présenter le secteur d'activité et le contexte organisationnel dans lequel s'inscrit le projet logiciel. Décrire les enjeux de la transformation numérique pour ce secteur : besoins d'automatisation, limites des outils existants (traitements manuels sur papier ou Excel, dispersion de l'information, absence de traçabilité...). Situer l'importance du développement logiciel comme réponse à ces enjeux. 1 à 2 paragraphes.

2. Problématique

Formuler clairement le problème auquel répond le projet. La problématique identifie les insuffisances du système existant et justifie la nécessité d'une solution logicielle. Elle peut se conclure par une question centrale. Exemple : « Comment concevoir et développer une application web permettant de centraliser et d'automatiser la gestion des demandes de stage, en garantissant un accès sécurisé selon les profils d'utilisateurs ? » 1 paragraphe.

3. Objectifs du rapport

Énoncer l'objectif général, puis 3 à 5 objectifs spécifiques numérotés. Exemples : (1) Analyser les besoins fonctionnels et non fonctionnels de la solution, (2) Modéliser le système avec UML/Merise, (3) Concevoir la base de données relationnelle, (4) Développer l'application avec les technologies choisies, (5) Tester et valider la conformité aux exigences.

4. Structure du rapport

Présenter brièvement le contenu de chaque chapitre en 2 à 3 lignes. Exemple : « Ce rapport est organisé en trois chapitres. Le premier chapitre présente la structure d'accueil et l'analyse de l'existant. Le deuxième chapitre expose la conception de la solution... »

CHAPITRE 1 : ETAT DE L'ART

CHAPITRE 1 : PRÉSENTATION DE LA STRUCTURE D'ACCUEIL ET ANALYSE DES BESOINS

Introduction du chapitre

Annoncer en 2 à 3 phrases le contenu et les objectifs de ce chapitre : présenter l'organisation d'accueil, analyser le système existant et recueillir les besoins auxquels la solution logicielle devra répondre.

1.1. Présentation de la structure d'accueil

1.1.1. Historique, missions et activités

Décrire l'organisation : dénomination, date de création, statut juridique, secteur d'activité, missions principales, taille (effectif, chiffre d'affaires si disponible), localisation géographique. Ces informations sont disponibles dans les documents institutionnels ou sur le site de la structure. 1 à 2 paragraphes.

1.1.2. Organisation interne

Insérer l'organigramme de la structure (à créer avec draw.io, Lucidchart ou Word SmartArt) et le commenter. Préciser le positionnement du service informatique ou du département en charge du développement. Mentionner les outils et technologies déjà en usage dans la structure (logiciels métier, ERP, CRM, outils bureautiques...).

1.1.3. Présentation du service d'accueil et missions du stagiaire

Décrire le service ou département dans lequel le stage s'est déroulé : missions du service, équipe, place du stagiaire dans l'équipe. Préciser les tâches qui ont été confiées à l'apprenant durant le stage et leur lien avec le projet de développement.

1.2. Analyse de l'existant

1.2.1. Description du système actuel

Décrire le système en place avant le projet : traitement manuel (papier, Excel, téléphone), logiciel déjà utilisé mais insuffisant, processus métier actuels. Illustrer par un schéma du processus actuel (diagramme de flux ou BPMN simplifié) si pertinent.

1.2.2. Critique de l'existant

Lister et analyser les dysfonctionnements, insuffisances et risques du système actuel. Structurer l'analyse en : (1) problèmes d'efficacité (lenteur, redondance, erreurs fréquentes), (2) problèmes de fiabilité (perte de données, incohérences), (3) problèmes d'accessibilité (accès limité, pas de consultation à distance), (4) problèmes de sécurité (absence de contrôle d'accès, données non protégées). Ce diagnostic justifie le projet de développement.

1.2.3. Solutions existantes sur le marché

Présenter brièvement les solutions logicielles existantes (solutions open source, SAAS, progiciels) qui pourraient répondre au besoin. Dresser un tableau comparatif : Solution | Fonctionnalités clés | Avantages | Inconvénients | Coût. Conclure en justifiant pourquoi le développement d'une solution sur mesure a été retenu.

1.3. Recueil et analyse des besoins

1.3.1. Identification des acteurs et parties prenantes

Lister tous les acteurs qui interagiront avec le futur système : utilisateurs finaux (employés, clients, étudiants...), administrateurs système, responsables métier, systèmes externes. Pour chaque acteur, préciser son rôle et ses attentes vis-à-vis de l'application.

1.3.2. Besoins fonctionnels

Lister les fonctionnalités que le système doit offrir. Structurer par module ou par acteur. Utiliser un tableau : ID | Acteur | Besoin fonctionnel | Priorité (Haute/Moyenne/Faible). Exemples pour une application de gestion : BF01 | Administrateur | Gérer les comptes utilisateurs (CRUD) | Haute ; BF02 | Utilisateur | S'authentifier avec login/mot de passe | Haute ; BF03 | Responsable | Générer des rapports statistiques | Moyenne.

1.3.3. Besoins non fonctionnels

Définir les contraintes de qualité du système. Les classer en : (1) Performance (temps de réponse $< 2s$, support de N utilisateurs simultanés), (2) Sécurité (authentification, gestion des rôles et permissions, chiffrement des données sensibles, protection contre les injections SQL et XSS), (3) Disponibilité et fiabilité (taux de disponibilité souhaité), (4) Maintenabilité (code documenté, architecture modulaire), (5) Compatibilité (navigateurs supportés, responsive design, OS).

1.3.4. Contraintes techniques et organisationnelles

Préciser les contraintes imposées par la structure d'accueil ou le contexte : budget alloué, délais de réalisation, technologies imposées ou proscrites, infrastructure existante à respecter (OS serveur, SGBD déjà en place), compétences disponibles dans l'équipe, contraintes réglementaires (RGPD, loi sur la protection des données personnelles).

Conclusion du chapitre

Résumer les besoins identifiés et faire la transition vers le chapitre de conception. Mentionner que l'analyse menée constituera le fondement de la conception de la solution. 3 à 5 lignes.

Introduction du chapitre

Présenter en 2 à 3 phrases l'objectif de ce chapitre : modéliser et concevoir la solution logicielle à partir des besoins recueillis au chapitre 1.

2.1. Choix de la méthodologie de développement

2.1.1. Présentation de la méthodologie retenue

Présenter la méthodologie de développement choisie parmi : cycle en V (adapté aux projets à exigences stables), méthode Agile/Scrum (itérations courtes, adaptation continue), méthode 2TUP (Two Track Unified Process, combinaison d'UML et de cycles itératifs), RAD (Rapid Application Development). Décrire les phases de la méthode et le calendrier du projet (diagramme de Gantt ou tableau des sprints).

2.1.2. Justification du choix

Justifier le choix de la méthodologie en fonction du contexte : taille du projet, stabilité des besoins, durée du stage, taille de l'équipe, niveau de maîtrise. Argumenter pourquoi les méthodes alternatives ont été écartées.

2.2. Modélisation avec UML

2.2.1. Diagramme des cas d'utilisation

Présenter le diagramme des cas d'utilisation (use case) global du système. Il doit faire apparaître tous les acteurs et tous les cas d'utilisation principaux. Insérer le diagramme réalisé avec StarUML, PlantUML, draw.io ou Visual Paradigm, numéroté en tant que Figure. Décrire ensuite sous forme de tableau les cas d'utilisation les plus importants : Nom du cas | Acteur | Description | Préconditions | Postconditions.

2.2.2. Diagrammes de séquence

Présenter les diagrammes de séquence pour les scénarios les plus représentatifs (authentification, création d'un enregistrement, génération d'un rapport...). Pour chaque diagramme, préciser le titre, les acteurs impliqués et les objets/composants concernés. Minimum : 3 diagrammes de séquence.

2.2.3. Diagramme de classes

Présenter le diagramme de classes du domaine métier. Il doit représenter les entités principales, leurs attributs (avec types), leurs méthodes et les relations entre elles (association, composition, héritage, dépendance). Le diagramme doit être cohérent avec le MCD de la section suivante. Insérer le diagramme numéroté en tant que Figure.

2.2.4. Diagramme d'activités

Présenter le ou les diagrammes d'activités pour les processus métier les plus complexes (workflow de validation d'une demande, processus de commande, cycle de vie d'un ticket...). Montrer les flux alternatifs et les décisions. Minimum : 1 diagramme d'activités.

2.2.5. Diagramme de déploiement

Présenter l'architecture de déploiement de l'application : serveur web, serveur de base de données, clients (navigateurs), éventuels services tiers (API externes, serveur de messagerie). Préciser les technologies de communication (HTTP/HTTPS, JDBC, REST...) et les ports utilisés.

2.3. Conception de la base de données

2.3.1. Modèle Conceptuel des Données (MCD)

Présenter le MCD (notation Merise) qui représente les entités du domaine métier, leurs attributs et les associations entre elles avec leurs cardinalités. Le MCD doit être cohérent avec le diagramme de classes UML. Insérer le schéma réalisé avec PowerAMC, JMerise, ou draw.io, numéroté en tant que Figure.

2.3.2. Modèle Logique des Données (MLD)

Présenter le MLD issu de la transformation du MCD. Montrer explicitement les règles de passage (entités $\rightarrow$ tables, associations $\rightarrow$ clés étrangères, associations $n,n \rightarrow$ tables de jonction). Format textuel : `NOM_TABLE (attribut1, #clé_étrangère, attribut2, ...)` où la clé primaire est soulignée.

2.3.3. Dictionnaire de données

Présenter le dictionnaire de données sous forme de tableau pour les tables principales : Nom du champ | Table | Type | Taille | Obligatoire | Description / Contrainte. Ce tableau est essentiel pour la documentation technique et facilite la maintenance future de la base.

2.4. Conception des interfaces

2.4.1. Charte graphique et ergonomie

Définir la charte graphique de l'application : palette de couleurs (codes hexadécimaux), typographie (polices choisies), style général (flat design, material design, bootstrap...). Justifier les choix en termes d'ergonomie et d'expérience utilisateur (UX). Mentionner le respect du design responsive si applicable.

2.4.2. Maquettes des interfaces principales

Présenter les maquettes (wireframes ou mockups) des pages principales de l'application : page de connexion, tableau de bord / accueil, formulaire de saisie principal, page de liste/consultation, page de rapport ou statistiques, page d'administration. Pour chaque maquette, insérer l'image numérotée et commenter brièvement la disposition des éléments et les interactions prévues. Les maquettes peuvent être réalisées avec Figma, Balsamiq, Adobe XD ou même papier-crayon scanné.

Conclusion du chapitre

Résumer les choix de conception effectués (méthodologie, modèles UML, structure de la BDD, interfaces) et faire la transition vers le chapitre d'implémentation. 3 à 5 lignes.

CHAPITRE 3 : IMPLÉMENTATION, TESTS ET RÉSULTATS

Introduction du chapitre

Présenter en 2 à 3 phrases l'objectif de ce chapitre : mettre en œuvre la conception réalisée au chapitre 2 et valider la solution par des tests rigoureux.

3.1. Environnement de développement

3.1.1. Matériel utilisé

Décrire la configuration matérielle du poste de développement : processeur, RAM, stockage, système d'exploitation. Si un serveur de développement a été utilisé, préciser ses caractéristiques. Présenter sous forme de tableau.

3.1.2. Logiciels et outils de développement

Lister et justifier le choix de chaque outil utilisé : IDE (VS Code, IntelliJ IDEA, Eclipse, NetBeans, PyCharm...), serveur web/application (Apache, Nginx, Tomcat, Node.js...), SGBD (MySQL, PostgreSQL, MariaDB, MongoDB, SQLite...), gestionnaire de dépendances (npm, pip, Maven, Composer...), outil de versioning (Git, GitHub, GitLab...), outil de modélisation (draw.io, StarUML, JMerise...), outil de maquettage (Figma, Balsamiq...), outil de test (Postman, JUnit, PHPUnit, Pytest...). Préciser les versions de chaque outil.

3.1.3. Technologies et langages retenus

Justifier le choix de chaque technologie : langage(s) de programmation (PHP, Python, Java, JavaScript, C#...), framework(s) back-end (Laravel, Django, Spring Boot, Express.js, Symfony...), framework(s) front-end (React, Vue.js, Angular, Bootstrap, Tailwind CSS...), ORM (Eloquent, Hibernate, SQLAlchemy...), protocole d'API (REST, GraphQL, SOAP...). Pour chaque choix, présenter un tableau comparatif avec les alternatives évaluées.

3.2. Implémentation de l'application

3.2.1. Architecture de l'application

Décrire l'architecture logicielle adoptée : MVC (Model-View-Controller), architecture en couches (présentation / métier / accès aux données), architecture microservices, architecture REST API + Single Page Application. Présenter un schéma de l'architecture applicative numéroté en tant que Figure. Expliquer comment les différentes couches ou composants communiquent entre eux.

3.2.2. Structure du projet

Présenter l'organisation des fichiers et répertoires du projet (arborescence). Expliquer le rôle de chaque dossier principal. Exemple pour un projet Laravel : /app (modèles, contrôleurs), /database (migrations, seeders), /resources (vues, assets), /routes (définition des routes), /config (configuration). Insérer une capture d'écran de l'arborescence dans l'IDE.

3.2.3. Implémentation des fonctionnalités clés

Décrire l'implémentation des fonctionnalités les plus importantes et représentatives du projet. Pour chaque fonctionnalité majeure : (1) rappeler brièvement la fonction, (2) décrire les choix d'implémentation, (3) insérer les extraits de code les plus significatifs (en police Courier New 10pt, sur fond légèrement grisé), (4) insérer une capture d'écran de l'interface correspondante. Traiter au minimum : le système d'authentification et gestion des rôles, le module CRUD principal, la gestion des relations entre entités, la génération de rapports ou exports (PDF, Excel), les fonctionnalités de recherche et filtrage. Éviter de coller tout le code : ne montrer que les extraits pertinents, le reste va en annexe.

3.2.4. Implémentation de la sécurité

Décrire les mesures de sécurité implémentées dans le code : hachage des mots de passe (bcrypt, Argon2 ...), protection CSRF (tokens), protection contre les injections SQL (requêtes préparées, ORM), protection XSS (échappement des sorties), gestion des sessions et tokens JWT, contrôle des accès par rôle (middleware, guards), validation des données en entrée (côté serveur). Illustrer chaque mesure d'un extrait de code.

3.2.5. Implémentation de la base de données

Décrire la mise en place de la base de données : création du schéma (script SQL de création ou migrations), insertion des données de référence (seeders), gestion des index pour optimiser les requêtes, contraintes d'intégrité référentielle. Insérer un extrait du script SQL ou des

migrations pour les tables principales. Présenter une capture d'écran du schéma de base de données dans l'outil de gestion (phpMyAdmin, pgAdmin, DBeaver...).

3.3. Tests et validation

3.3.1. Stratégie de tests

Présenter la stratégie de tests adoptée. Définir les types de tests réalisés : tests unitaires (fonctions et méthodes individuelles), tests d'intégration (interaction entre composants), tests fonctionnels (validation des cas d'utilisation), tests de sécurité (tentatives d'injection, accès non autorisés), tests de performance (temps de réponse, charge), tests d'acceptance/utilisateur (UAT). Préciser les outils utilisés pour chaque type de test.

3.3.2. Plan et résultats des tests fonctionnels

Présenter le plan de tests fonctionnels sous forme de tableau : ID test | Cas d'utilisation testé | Données d'entrée | Résultat attendu | Résultat obtenu | Statut (PASS/FAIL). Couvrir au minimum les scénarios nominaux et alternatifs pour chaque fonctionnalité principale. Illustrer les tests réussis et échoués avec des captures d'écran. Pour les tests échoués, décrire le bug identifié et la correction apportée.

3.3.3. Tests unitaires automatisés

Si des tests unitaires ont été automatisés (JUnit, PHPUnit, Pytest, Jest...), présenter le taux de couverture du code (code coverage), insérer une capture d'écran du rapport de tests et commenter les résultats. Si les tests automatisés n'ont pas pu être mis en place, le justifier et décrire la procédure de tests manuels utilisée.

3.3.4. Tests de performance

Si des tests de performance ont été réalisés (avec Apache JMeter, Locust, Artillery, ou simplement via le chronomètre), présenter les résultats : temps de chargement des pages principales, temps de réponse des requêtes critiques, comportement sous charge simulée. Présenter les résultats sous forme de tableau ou graphique.

3.4. Présentation de l'application finale

Présenter l'application finale à travers une série de captures d'écran commentées des interfaces principales : écran de connexion, tableau de bord, pages de gestion (liste, formulaire de création, formulaire de modification, confirmation de suppression), génération de rapport, interface d'administration. Pour chaque capture, rédiger une légende explicative. Ces captures constituent la preuve du travail réalisé.

Conclusion du chapitre

Faire le bilan de l'implémentation : fonctionnalités réalisées vs planifiées, résultats des tests, problèmes rencontrés et solutions apportées. Faire la transition vers le chapitre de discussion. 3 à 5 lignes.

CHAPITRE 4 : DISCUSSION, BILAN ET PERSPECTIVES

CHAPITRE 4 : DISCUSSION, BILAN ET PERSPECTIVES

4.1. Résumé des réalisations

Dresser un bilan synthétique de l'ensemble du projet. Utiliser un tableau récapitulatif : Objectif fixé | Réalisation | Statut (Atteint / Partiellement atteint / Non atteint) | Commentaire. Ce tableau montre clairement l'adéquation entre les objectifs initiaux et les livrables effectifs.

4.2. Analyse critique de la solution

4.2.1. Points forts

Identifier et argumenter les points forts de la solution développée : couverture fonctionnelle complète des besoins exprimés, qualité de l'interface utilisateur (ergonomie, esthétique, responsive), robustesse et sécurité de l'application, qualité du code (lisibilité, modularité, respect des conventions), performance des requêtes et des pages, facilité de déploiement et de maintenance.

4.2.2. Limites et points d'amélioration

Identifier honnêtement les limites de la solution : fonctionnalités prévues mais non implémentées (faute de temps), limitations techniques (pas d'API mobile, pas de notifications en temps réel, pas d'export Excel avancé...), qualité de code perfectible (absence de tests unitaires complets, dette technique...), limitations de performance (non testé en conditions réelles de charge), contraintes de déploiement. Cette section montre la maturité et l'esprit critique de l'apprenant.

4.3. Perspectives et recommandations

Proposer des axes d'amélioration et d'évolution concrètes pour la version suivante de l'application : développement d'une application mobile native (Flutter, React Native), intégration d'une API REST pour l'interopérabilité avec d'autres systèmes, mise en place d'un module de notifications (email, SMS, push), intégration de tableaux de bord analytiques avancés (Chart.js, D3.js), déploiement sur le cloud (AWS, Azure, OVH), mise en place d'une architecture microservices pour la scalabilité, intégration de l'intelligence artificielle

(recommandations, prédictions). Formuler des recommandations concrètes à la structure d'accueil pour l'évolution et la maintenance de l'application.

4.4. Bilan personnel du stage

L'apprenant exprime son retour d'expérience sur le stage et le projet : compétences techniques acquises ou renforcées (langages, frameworks, outils, méthodologies), compétences transversales développées (gestion de projet, travail en équipe, communication avec les utilisateurs, rédaction de documentation technique), difficultés rencontrées et comment elles ont été surmontées, écart éventuel entre la formation académique et les exigences professionnelles, apport de ce stage pour la définition du projet professionnel.

CONCLUSION GENERALE

Résumé du Chapitre 1

Rappeler en 2 à 3 phrases les principaux éléments du chapitre 1 : présentation de la structure, critique de l'existant et besoins identifiés.

Résumé du Chapitre 2

Rappeler en 2 à 3 phrases les choix de conception : méthodologie, modèles UML réalisés, structure de la base de données, maquettes des interfaces.

Résumé du Chapitre 3

Rappeler en 2 à 3 phrases les technologies utilisées, les fonctionnalités implémentées et les résultats des tests.

Résumé du Chapitre 4

Rappeler en 2 à 3 phrases le bilan des réalisations, les limites identifiées et les perspectives de développement proposées.

Impact et apport du travail

Exprimer l'impact de la solution développée pour la structure d'accueil (gain de temps, réduction des erreurs, meilleure traçabilité, satisfaction des utilisateurs) et l'apport académique du projet. 1 paragraphe.

Mot de fin

Clôturer le rapport en quelques lignes d'ouverture : défis futurs du génie logiciel (IA générative dans le code, développement low-code/no-code, DevOps/CI-CD, cybersécurité applicative, cloud-native development), motivation de l'apprenant à poursuivre dans ce domaine. 3 à 5 lignes.

BIBLIOGRAPHIE

Lister toutes les références utilisées, classées par ordre alphabétique du nom d'auteur, selon la norme APA ou ISO 690.

Distinguer les catégories suivantes.

Ouvrages : GAMMA E., HELM R., JOHNSON R., VLISSIDES J. – Design Patterns : Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

Articles scientifiques : auteur(s), titre, nom de la revue, volume, numéro, pages, année.

Sites web et documentations officielles : auteur si disponible, titre de la page, URL complète, date de consultation entre parenthèses.

Documentations techniques : frameworks (Laravel, Django, Spring...),

RFC, spécifications UML de l'OMG, documentations SGBD.

Cours et supports académiques : NOM Prénom de l'enseignant, Intitulé du cours, Établissement, Année.

Minimum attendu : 12 références.

ANNEXES

Numéroter chaque annexe (Annexe A, Annexe B...) et lui donner un titre explicite. Référencer chaque annexe dans le corps du rapport (ex : « voir Annexe B »).

Annexe A : Code source des modules principaux

Insérer les extraits de code source les plus importants qui n'ont pas pu être intégrés dans le corps du rapport : contrôleurs principaux, modèles de données, fichiers de configuration (sans données sensibles), scripts de migration de base de données. Police Courier New, taille 10, fond légèrement grisé.

Annexe B : Scripts SQL de création de la base de données

Insérer le script SQL complet de création des tables, des index, des contraintes d'intégrité et des données de référence (INSERT INTO pour les tables de paramétrage). Police Courier New, taille 10.

Annexe C : Manuel utilisateur

Rédiger un guide d'utilisation destiné aux utilisateurs finaux non techniciens. Pour chaque module de l'application : décrire la procédure étape par étape, illustrer chaque étape d'une capture d'écran annotée, mentionner les erreurs fréquentes et leur solution. Ce manuel peut être fourni à la structure d'accueil comme documentation d'utilisation.

Annexe D : Manuel d'installation et de déploiement

Décrire la procédure complète d'installation et de déploiement de l'application : prérequis matériels et logiciels (OS, versions des dépendances), étapes d'installation (clonage du dépôt, installation des dépendances, configuration du fichier .env, création et migration de la base de données, démarrage du serveur), procédure de mise à jour, conseils de sauvegarde. Ce document est indispensable pour la maintenance.

Annexe E : Glossaire technique

Définir les termes techniques spécifiques au projet ou au domaine métier utilisés dans le rapport. Format : Terme : Définition. Distinguer les termes informatiques des termes métier propres à la structure d'accueil.

TABLE DES MATIERES

*La table des matières détaillée (niveaux 1 à 3) est générée automatiquement par Word :
Références → Table des matières → Insérer une table des matières, niveaux 1 à 3.*

*À mettre à jour avant l'impression finale : clic droit sur la table → Mettre à jour les champs →
Mettre à jour toute la table.*

NB : L'ensemble du document est rédigé avec la police Times new Roman. Taille 14. Interligne 1,5. Justifié.